Actividad 3.1. Analizador léxico

Estefanía Antonio Villaseca A01736897

Miranda Eugenia Colorado Arróniz A01737023

Alejandro Kong Montoya A01734271

Restituto Lara Larios A01737216

24 de abril de 2026

Desarrollo de aplicaciones avanzadas de ciencias computacionales (Gpo 501)

Prof. Luciano García Bañuelos

Para el desarrollo del analizador léxico se tomó como referencia el estándar ISO/IEC 9899:1999 (C99), pero no se intentó cubrir la totalidad del lenguaje, sino un fragmento representativo y suficiente para analizar programas escritos en C de complejidad básica e intermedia. La selección implementada en *lexer.py* se enfoca en los elementos léxicos más frecuentes y necesarios para reconocer la estructura general de programas fuente reales, especialmente aquellos que contienen declaraciones, definiciones de funciones, expresiones, estructuras de control y directivas sencillas del preprocesador.

Asimismo, el fragmento seleccionado incluye palabras reservadas del lenguaje C, identificadores, constantes numéricas, literales de cadena, constantes de carácter, comentarios, signos de puntuación y operadores. También se añadió soporte parcial para elementos del preprocesador, como números de preprocesamiento *PP_NUMBER*, otros símbolos no clasificados *PP_OTHER* y nombres de encabezado *HEADER_NAME* en casos de *#include*. De este modo, esta decisión permite que el analizador sea útil no solo para ejemplos sencillos, sino también para archivos fuente que se acerquen más a la escritura real de programas en C.

Dentro del alcance definido para el proyecto, varios aspectos del analizador sí se implementan de manera consistente con C99. En particular, se distingue entre tokens de preprocesamiento y tokens del lenguaje propiamente dicho, siguiendo la separación conceptual planteada en la sección 6.4 del estándar. También se respeta la clasificación básica de elementos léxicos en categorías como palabras reservadas, identificadores, constantes, literales de cadena y signos de puntuación. De la misma forma, el analizador reconoce las palabras reservadas de C99 con distinción entre mayúsculas y minúsculas, trata los identificadores con una estructura compatible con la especificación general del estándar, y aplica una estrategia coherente con la regla de la coincidencia más larga en casos como números de preprocesamiento y operadores compuestos. Además, se reconocen puntuadores definidos por el lenguaje, como *<:, :>*, *<%, %>*, *%:* y *%:%:*. En este sentido, aunque el analizador no cubre todo el estándar, sí reproduce varios de sus comportamientos léxicos fundamentales de forma fiel en los casos contemplados.

Por otra parte, la razón principal para seleccionar este subconjunto es que representa la parte más didáctica y funcional del análisis léxico, ya que con estos componentes es posible identificar correctamente la mayoría de los tokens que aparecen en programas comunes, sin entrar todavía en las partes más complejas del estándar que normalmente requieren varias fases de traducción, reglas contextuales adicionales o una integración más fuerte con el análisis sintáctico y semántico.

No obstante, no se implementó el estándar completo de C99 porque varias de sus características aumentan considerablemente la complejidad del analizador y no eran indispensables para el objetivo del proyecto. Por ejemplo, no se implementaron todas las fases de traducción definidas por el estándar, es decir, no se modelan de forma completa procesos como el reemplazo total de secuencias especiales previas al análisis léxico o todas las transformaciones previas del preprocesador. De igual manera, tampoco se implementó un preprocesador completo, ya que el archivo *lexer.py* reconoce ciertos elementos relacionados con preprocesamiento, pero no realiza expansión de macros, compilación condicional, sustitución de identificadores definidos con *#define*, ni evaluación de directivas como *#if*, *#ifdef*, *#elif* o *#undef*.

En la misma línea, el tratamiento de literales se dejó en un nivel práctico y no exhaustivo, ya que, aunque se reconocen constantes enteras, flotantes, cadenas y caracteres, no se cubren

todas las variantes posibles de prefijos y combinaciones permitidas por el estándar en todos los contextos. Además, algunos detalles finos del comportamiento definido por C99 para literales se manejan de manera simplificada. En particular, la concatenación de literales de cadena adyacentes no reproduce completamente en todos los casos el comportamiento especificado por el estándar durante las fases de traducción, especialmente cuando intervienen secuencias de escape cuyo significado debe resolverse antes de la concatenación. Del mismo modo, el reconocimiento de literales de cadena y constantes de carácter es permisivo en ciertos casos, por lo que algunas secuencias de escape inválidas podrían ser aceptadas sin emitir el diagnóstico formal que exigiría C99.

De forma similar, aunque se incorporó soporte básico para nombres universales de carácter y para escapes octales y hexadecimales, no se implementan todas las restricciones léxicas específicas establecidas por el estándar para estas construcciones. Por ello, la validación de secuencias como *\u*, *\U*, *\x* y algunos escapes octales no es completamente exhaustiva con respecto a los límites y condiciones formales definidos por C99.

Además, no se incluyeron validaciones semánticas profundas, puesto que el analizador léxico solo clasifica tokens y no verifica, por ejemplo, si un identificador fue declarado antes, si un tipo es válido en cierto contexto, o si una construcción tiene sentido dentro de la gramática del lenguaje, ya que eso corresponde más al análisis sintáctico y semántico que al léxico. Del mismo modo, no se resolvieron todos los casos contextuales del lenguaje, debido a que en C existen situaciones donde la interpretación final de un token depende del contexto sintáctico o semántico, como la diferencia entre un identificador común y un nombre de tipo definido por typedef, o entre un identificador y una constante de enumeración, por lo que ese nivel de decisión no forma parte del alcance de este lexer.

Finalmente, el manejo de errores se mantuvo simple, de manera que si aparece un símbolo no reconocido, el lexer avanza y continúa el análisis, pero no construye un sistema detallado de recuperación ni diagnósticos avanzados. En este sentido, estas exclusiones se justifican porque el objetivo del trabajo no es construir un compilador completo de C99, sino diseñar e implementar un analizador léxico comprensible, funcional y coherente con lo visto en clase, ya que incluir todas las reglas del estándar haría el proyecto mucho más largo y menos claro desde el punto de vista didáctico. En cambio, el subconjunto elegido permite demostrar correctamente los principios del análisis léxico, como el reconocimiento de patrones, la clasificación de tokens y la preparación de la entrada para etapas posteriores del compilador.

Además, el archivo *lexer.py* ya refleja esta intención, pues reconoce los componentes centrales del lenguaje, organiza los tokens de forma clara y maneja algunos casos especiales útiles, como la combinación de nombres de encabezado y la diferenciación entre constantes normales y elementos de preprocesamiento. Por ello, se considera que la selección realizada es adecuada para los objetivos del ejercicio, ya que equilibra fidelidad al estándar, viabilidad técnica y claridad académica.

En conclusión, el analizador desarrollado toma un fragmento suficientemente representativo de C99 para cumplir con el propósito del proyecto, ya que la selección se centró en los elementos léxicos esenciales del lenguaje y dejó fuera aquellas partes cuya complejidad pertenece más al preprocesamiento completo o a etapas posteriores del análisis.